Predict, Don't Just Pair: Lossless Storage for Families of Fine-Tuned Models

Anonymous Author(s)

Abstract

Model hubs are versioned stores of millions of near-identical fine-tuned checkpoints. Lossless systems for them treat storage mainly as a *pairing* problem, deciding which stored tensor each new tensor should be delta-encoded against. We revisit this view with exhaustive ground truth. For 130 fine-tunes of four model families (817 GB after deduplication) we compress every candidate tensor pair and compute the exact optimum of base selection. Near-optimal pairing is easy to reach. Removing one filter from the state-of-the-art online planner closes gaps of up to 16.9 points to the one-hop optimum. Most of the remaining redundancy lies in how the residual is *predicted* and *coded*. We present Tessera, a lossless tensor store built on three ideas. First, an entropy coder conditions every residual symbol on the exponent of the base weight, which the decoder already knows. Second, fine-tunes into which low-rank adapters were merged are predicted by a low-rank update whose decoding kernel is bit-reproducible across machines. Third, hop limits follow download popularity, so rarely used models may chain two deltas deep, and we solve the resulting recreation-bounded storage problem exactly. Planned offline with exact costs, Tessera stores 25.3% of the deduplicated bytes, versus 47.6% for our re-implementation of the TensorDex design and 36.3% for the original ZipLLM. Its 74.7% reduction is 14 points above the exact optimum of pairing alone, with the same hop limits or any chain length. At equal download-weighted read cost, its plans store 0.9–3.7 points more than the greedy (LMG) and Prim-style (MP) heuristics of dataset versioning. Replayed online on three families, the store ends within 0.2–1.4 points of the offline plan, ingests about 1.8 GB/s from local disk, and reconstructs a 6 GB model in 1.5–3.7 s on a CPU and 0.05–0.12 s on a GPU.

CCS Concepts

• **Information systems** → **Storage architectures**; **Data compression**.

Keywords

model storage, data versioning, lossless compression, delta compression, entropy coding, model hubs

1 Introduction

Open model hubs have become the package registries of machine learning. TensorDex reports that Hugging Face's total storage doubled in five months and is heading toward the exabyte scale [26]. Fine-tuned derivatives dominate this storage. A few base models spawn thousands of variants for instruction following, safety, coding, other languages, or a single user's task, and each variant is uploaded as a full copy of the weights even though it differs only slightly from its relatives. A hub is, in effect, a versioned store of large binary objects, and deciding which versions to store as deltas of which others is the recreation/storage tradeoff studied for dataset versioning [7]. What differs is the scale and the content, floating-point tensors whose deltas are neither sparse nor byte-aligned.

Two lines of systems work exploit this redundancy losslessly. Single-model coders (ZipNN, DFloat11) entropy-code the skewed exponents of floating-point weights and save about a third of the bytes [17, 44]. Cross-model *delta* systems store a tensor as its difference from a similar stored tensor and then compress that residual [26, 35, 42]. The most recent of these, TensorDex [26], argues that redundancy lives at tensor granularity. It sketches every tensor, predicts pairwise compressibility, and runs an online clustering planner (FlexSplit) that chooses a base for every tensor. In this view, storage reduction is mainly a *pairing* problem. Once each tensor has the right base, its XOR or arithmetic residual is handed to a generic entropy coder.

This paper asks how much pairing matters. How far do practical planners fall from the optimal pairing, and how much redundancy does the best possible pairing still leave behind? Answering them requires ground truth that prior evaluations lacked. For four model families (Qwen2.5-3B, Llama-3.2-3B, Gemma-2-2B, Qwen2.5-7B; 130 fine-tunes, 817 GB after deduplication), we compress *every* candidate pair of same-named tensors, over a million directed pairs in total, and solve the one-hop base-selection problem exactly as an integer program per tensor group. Our findings (§3) run against the pairing view:

- **Near-optimal pairing is within reach online, but FlexSplit misses it by up to 16.9 points.** Our re-implementation of FlexSplit trails the exact one-hop optimum by 0 to 16.9 points, depending on family structure. The gap comes from a candidate filter in its cluster-splitting phase, which never promotes the natural center of a sub-family, such as an Instruct model whose many fine-tunes sit at the cluster mean. Removing the filter brings FlexSplit within 0.2 points of the optimum in the three smaller families and within 1.0 on Qwen-7B.
- **Tensor granularity adds little within a family.** The best tensor-level pairing beats the best model-level pairing by 0.3–1.0 points.
- **Pairing barely depends on the codec.** On Qwen, re-optimizing the pairing for the codec of §4.2 changes the result by 0.06 points.

Even the optimal pairing therefore leaves the bulk of the redundancy unused (Figure 1). The exact one-hop optimum of pairing reduces storage by 59.5%, and allowing chains of any length raises it only to 60.6%. The remaining gains come from *what is stored for a pair*. We build Tessera (Figure 2) around three observations about fine-tuned weights:

SIGMOD '27, Huntington Beach, CA, USA
2026.

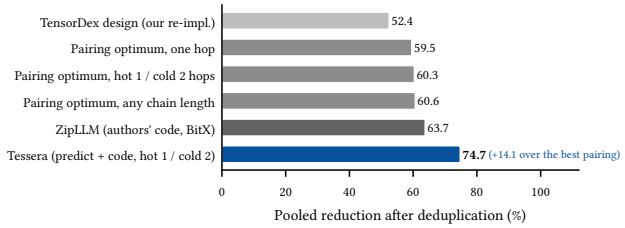


Figure 1: Pooled reduction after deduplication over the four families. Pairing-only designs with the XOR/zstd codec stay at or below 60.6%, even with exact optima and chains of any length, and ZipLLM with its byte-plane codec reaches 63.7%. Predicting and coding the residual from the base takes Tessera to 74.7%.

(1) **The base predicts the residual's distribution.** Fine-tuning moves a weight by an amount that scales with the weight itself. The decoder already knows the base, so Tessera codes an order-preserving integer residual with rANS [10], using a separate frequency table for each exponent of the base weight. This base-conditioned codec adds 10–11 points in every family (§4.2).

(2) **Many fine-tunes are low-rank updates.** LoRA-merged models [21] differ from their parent by a product $US \cdot V$ of rank 4–32. Tessera detects this from a row sample, fits the factors, and stores them together with the small remaining residual. The decoder recomputes the prediction with a kernel that produces bit-identical results on any machine and thread count, as lossless storage requires. This adds 2.5–6.6 points (§4.3).

(3) **Downloads are highly skewed.** In our Qwen corpus, the top 20% of models receive 99.7% of the downloads. Tessera therefore limits hot models to one hop, as TensorDex does, and lets cold models chain two deltas deep, solving the resulting hop-bounded problem exactly. On the Qwen family this stores 1.2 points more at the same download-weighted read cost (§4.4).

Planned offline with exact costs, these three techniques raise the pooled reduction from 52.4% for our re-implementation of the TensorDex design to 74.7% (47% fewer bytes), with every tensor decoded along its chain and hash-verified. With the same hop limits, pairing alone reaches 60.3%, and ZipLLM, run with its authors' code, 63.7%. Tessera is an online store that ingests models in parallel, compacts groups transactionally, deletes models, and reconstructs them on CPUs and GPUs (§5). At the same download-weighted read cost, its exact hop-bounded plans store 0.9–3.7 points more than the storage/recreation heuristics of dataset versioning, with chains of two instead of up to 11 (§6.3). When each family is replayed online, the store ends within 0.2–1.4 points of the offline plan. The gap comes from how often it re-plans and from its plain-delta cost estimates, not from the search (§6.5).

Contributions.

- A ground-truth methodology for delta-based model storage, with all-pairs real compression and exact optima under hop bounds, applied to four model families (§3).
- An analysis of pairing against exact optima, including a change to one filter of FlexSplit that brings it within 0.2–1.0 points of the one-hop optimum (§3).
- A base-conditioned rANS codec, a low-rank predictive path with a bit-reproducible decoder, and popularity-aware hop limits solved exactly, which together reach a pooled lossless reduction of 74.7% (§4, §6.1–6.3).
- Tessera, an online tensor store with parallel ingestion, transactional compaction, deletion, and verified CPU/GPU retrieval, evaluated online, in a mixed-family stream, and on a random sample of the hub, where it reaches 76.4% (§5, §6.4–6.5).

2 Background

Checkpoints and tensor groups. A checkpoint is a set of named tensors, usually stored in BF16 (1 sign, 8 exponent, 7 mantissa bits). Fine-tunes of one base share the architecture, so tensors with the same name, dtype, and shape form a *group*. In the lossless systems we study, a tensor can only be encoded against a member of its own group. Exact duplicates from re-uploads, frozen layers, and untouched norms are frequent. Content hashing removes them first (*TensorID* in TensorDex), and all reduction ratios in this paper are measured against the bytes left after this deduplication, as in TensorDex.

Delta compression. A delta coder stores a tensor t as a residual against a base b from its group. The residual is either the bitwise XOR $t \oplus b$ (BitX in ZipLLM, TensorX in TensorDex) or an integer difference of order-preserving encodings of the floats (FM-Delta [35]). A generic byte-oriented compressor then codes the residual, in the systems above zstd [9] or Huffman coding. For close relatives the residual is mostly zeros in the high bits, which is why delta coding roughly doubles the savings of single-model coding (§3). Let a_j be the size of j coded standalone as a *base*, and D_{ij} its size coded against base i.

Problem. Groups are independent, so consider one group with distinct members V. A *plan* gives every $j \in V$ a parent $p(j) \in V \cup \{r\}$, where $p(j) = r$ means that j is stored standalone, such that the parent pointers form a tree rooted at r. It stores $\sum_j c(p(j), j)$ bytes, with $c(r, j) = a_j$ and $c(i, j) = \min(D_{ij}, a_j)$. Reconstructing j reads every blob on its path to r, and its *depth* is the number of deltas on that path. Given a hop limit h_j per tensor, we seek a minimum-storage plan in which every j has depth at most h_j. With $h_j = 1$ (*one hop*) this is uncapacitated facility location, which is NP-hard in general but small per group, since a group of N models has $N(N-1)$ directed candidate pairs. With $h_j = \infty$ it is a minimum-cost arborescence [11]. Limits in between trade storage for read cost, as in dataset versioning [7].

The TensorDex design. TensorDex [26] makes three choices that we revisit. (i) *TensorSketch*, a CountSketch [8] over the bits of a tensor (8 KB per tensor), estimates the normalized Hamming distance $\hat{p}$ between two tensors. (ii) *TensorPred*, a four-parameter regression on $\hat{p}$ and its binary entropy, predicts the reduction ratio of a pair. (iii) *FlexSplit* plans online. Phase I attaches each arriving

Table 1: Corpus. Raw sizes before and after content-hash deduplication.

Family	Models	Groups	Raw	Deduplicated
Qwen2.5-3B	40	434	246.9 GB	187.5 GB
Llama-3.2-3B	30	254	192.8 GB	145.8 GB
Gemma-2-2B	30	288	156.9 GB	108.9 GB
Qwen2.5-7B	30	339	456.9 GB	374.8 GB
Total	130	1,315	1,053.4 GB	816.9 GB

tensor to the existing base with the highest predicted ratio, or makes it a new base. Phase II periodically revisits every cluster. It considers as split candidates only the members whose ratio falls "significantly below the cluster average", promotes the candidate with the largest positive gain, and re-assigns the members that improve. The planned pairs are encoded with TensorX, an XOR-based codec, or FM++, an arithmetic coder. TensorDex reports 65.1% (TensorX) and 70.5% (FM++) reduction on 2,890 Hugging Face models.

Metrics. *Reduction* is 1 − stored/raw, with raw measured after deduplication. *Read amplification* of a model is the number of bytes read to reconstruct it, summing every blob on the chain of each of its tensors, divided by its raw size. Values below 1 mean that fewer bytes are read than the raw model holds. We weight read amplification by downloads, because a hub serves popular models far more often than cold ones.

3 Revisiting Pairing with Ground Truth

3.1 Methodology

Corpus. We study four model families (Table 1): Qwen2.5-3B [38], Llama-3.2-3B [32], Gemma-2-2B [12], and, to check scale, Qwen2.5-7B. The online experiments use the three smaller ones. For each family we take the most-downloaded full-weight fine-tunes of the base and instruct models from Hugging Face, with at most three repositories per organization. We keep only repositories whose safetensors metadata reports pure BF16 and exactly the parameter count of the reference model. Gated official roots are replaced by byte-identical public mirrors. Models are ingested in creation order, and popularity is the hub's download count at selection time. This selection favours homogeneous families of popular fine-tunes. Randomly sampled corpora, as in TensorDex and ZipLLM, mix more lineages, and §6.5 evaluates such a sample.

Exhaustive ground truth. Evaluations of delta systems usually measure a planner end to end, which cannot separate a bad plan from an inherent limit. We instead encode *every* ordered pair of same-group tensors with the codec under study, as well as every tensor standalone. Each plan's storage is then an exact sum of real encoded sizes. On the Qwen family this means 434 groups, each with 780 XOR pairs (the XOR residual is symmetric) or 1,560 directed pairs for our base-conditioned codec. The XOR pass takes 31 minutes on 160 processes. With these costs we solve one-hop base selection exactly, with one facility-location integer program per group solved by HiGHS [23]. For k-hop limits we use a layered integer program, and for unbounded chains the minimum spanning arborescence [11]. Unless noted otherwise, results in this section use the codec of TensorDex's TensorX (XOR residual, byte planes, zstd), and the reduction is measured on the Qwen family after deduplication (187.5 GB).

Table 2: Planners on the Qwen family with the XOR/zstd codec (reduction after deduplication; 187.5 GB). All numbers are real encoded sizes.

Planner	Reduction
Standalone coding (no deltas)	28.6%
Every model against the pretrained base	52.6%
FlexSplit (TensorSketch + TensorPred)	56.5%
FlexSplit with exact costs	57.4%
Declared `base_model` metadata (ZipLLM)	62.4%
Model-level one-hop optimum	62.4%
FlexSplit, all members as split candidates	**62.8%**
Tensor-level one-hop optimum	63.0%
Greedy multi-hop chains (depth ≤ 6)	64.7%

Re-implementation. TensorDex's code was not available when we started, so we re-implemented TensorSketch, TensorPred, and FlexSplit from the paper. For three details the paper leaves open, we chose the threshold for "significantly below the cluster average" (0.05), the Phase II period (every five arrivals), and the entropy stage of TensorX (zstd), and we fit TensorPred's coefficients on held-out models of our corpus. Our FlexSplit numbers therefore describe the design as we implemented it, not the authors' system. Their reported numbers come from a different corpus and are not comparable to ours.

3.2 Findings

Finding 1: deltas and pairing matter. Delta coding more than doubles the savings of standalone coding (28.6% → 63.0%), and the choice of base matters (Table 2). Encoding every model against the pretrained base reaches only 52.6%. Declared lineage is also unreliable. Four of the 40 repositories copy the Instruct model byte for byte but declare the pretrained base as their parent. These results confirm observations 1–2 of TensorDex.

Finding 2: FlexSplit's split filter misses the natural centers. Our FlexSplit trails the one-hop optimum by 6.5 points on Qwen, 16.9 on Llama, and 5.6 on Qwen-7B. On Gemma it matches the optimum (Table 3). Exact costs recover only 0.9 of the 6.5 points, so prediction error is not the cause, and disabling Phase II improves the result by 0.5 points, so the splits do not help. We traced the gap on individual groups. The pretrained model arrives first and becomes the base of one large cluster. The Instruct model and its many fine-tunes attach to it, because they are closer to the pretrained model than to any other base *present at their arrival*. The fine-tunes of Instruct sit *at* the cluster's average ratio, not below it, so the filter never proposes Instruct, the natural center, as a candidate, and no step demotes the first base. In one group this leaves the planner 15.8% above optimal, and lowering the threshold to 0 still leaves 4.8%. Allowing *every* member as a split candidate fixes this. The gap shrinks to 0.2 points on Qwen and Llama, 0.0 on Gemma, and 1.0 on Qwen-7B, while the planner stays online. A base-set local search with add, drop, and swap moves [6] closes another 0.1 points (0.9 on Qwen-7B), but it re-encodes five times as many tensors. The

Gemma family is flat. Nearly every fine-tune derives directly from gemma-2-2b-it, which also arrives first, so no base needs promoting.

Finding 3: tensor granularity adds little within a family. The best tensor-level pairing beats the best model-level pairing, one base model per target model, by 0.6 (Qwen), 0.3 (Llama), 1.0 (Gemma), and 0.7 points (Qwen-7B). For the average Qwen target model, 83% of its tensors have their best base in one single other model. Per-tensor choice may matter more for heterogeneous candidate sets, but among the fine-tunes of one family the model-level choice is almost as good.

Finding 4: sketch-based cost prediction is noisy, but good enough to rank. TensorSketch at TensorDex's configuration (depth 2, width 1024) picks the base that exact Hamming distance would pick for 76% of tensors (TensorDex reports ≈100%). The misses are near-ties, costing 0.10 points on average. Exact Hamming distance itself finds the truly smallest delta only 87% of the time, since the number of flipped bits is not the compressed size. TensorPred's error on held-out models is 3.6 points (TensorDex: 1.1), and it stays at 3.5 points with exact Hamming distances. The error therefore comes from the one-feature model, whose scatter splits into branches by tensor type. A predictor that entropy-codes the delta of the same 256K sampled elements of both tensors ranks bases better, with regret dropping from 0.50 to 0.05 points on Qwen, 0.06 to 0.005 on Llama, and 0.12 to 0.012 on Gemma, and on Llama and Gemma it reaches the exact one-hop optimum. We use it in Tessera.

Finding 5: pairing is robust to the codec. When we switch to the codec of §4.2, the pairing chosen under XOR/zstd stays within 0.06 points of the pairing re-optimized for the new codec (Qwen). Low-rank-aware re-planning (§4.3) likewise changes the one-hop result by 0.02 points. The best partners stay the same, and only their costs change.

Implication. In the pairing-only design space, the best any planner can reach is 63.0% (Qwen), 61.6% (Llama), 56.2% (Gemma), and 57.8% (Qwen-7B), and FlexSplit with the fix above gets within 0.2 points (1.0 on Qwen-7B, 0.1 with local search). The remaining 37–44% of the bytes are residuals of well-chosen pairs, and reducing them requires changing how residuals are *predicted* and *coded*.

4 Design of Tessera

Tessera keeps the tensor-group structure and one-hop default of TensorDex. It changes four things (Figure 2): how bases are chosen (§4.1), how a residual is coded given its base (§4.2), what is predicted before coding (§4.3), and how many hops a model may use (§4.4).

4.1 Choosing bases

Tessera plans online, as FlexSplit does, with two changes motivated by §3.

Sample-based cost prediction. When a tensor is ingested, Tessera stores a row sample of it, using the same rows for every member of the group and at most 32K elements. The cost of coding tensor j against candidate i is estimated by running the codec's histogram stage on the two samples and scaling the entropy of the resulting symbol streams to the full tensor. Unlike a sketch, this estimator is *codec-aware*, because it measures the symbols the coder will see. It therefore stays correct when the codec changes and needs no regression fit. It never decodes a stored tensor and costs one pass over two small samples per candidate.

Split candidates. A new tensor joins the candidate with the lowest estimated cost, or becomes a base if coding it standalone is cheaper. Periodically, Tessera re-plans each group, and every member is a candidate to become a new base, not only members that fall below the cluster average. The re-plan is a base-set local search (add, drop, swap) over the sampled costs. The guarantees known for it [6] assume metric costs, which ours are not, so §6.6 measures its gap. Re-planning is a transaction per group (§5). The new plan is encoded next to the old blobs and committed only if the group's total size shrinks. Because the re-plan is one-hop, it would also flatten two-hop chains of cold models. Tessera therefore keeps a tensor's current parent whenever that is predicted to be at least as cheap and the hop limits allow it, and it attempts a group only if the predicted gain of the remaining changes is at least 0.5% (§6.4).

4.2 Base-conditioned residual coding

Generic compressors code a residual as if it were independent of its base, but it is not. Fine-tuning moves a weight by an amount that grows with the weight's magnitude, so the base weight's exponent predicts how large the residual is. Because the decoder reconstructs the base first, Tessera can use the base as context at no storage cost.

Residual. Each BF16 value v is mapped to an integer that preserves the order of the floats, a bijection on all 2^{16} bit patterns: $o(v) = v$ if the sign bit is clear, and $o(v) = -(v \wedge \texttt{0x7FFF}) - 1$ otherwise. The residual $d = o(t) - o(b)$ is small whenever $t \approx b$, including across exponent boundaries, where XOR flips many bits. It is zigzag-mapped to $z < 2^{17}$ and split into a low byte $z \wedge \texttt{0xFF}$ and a high part $z \gg 8$.

Contexts. Both symbols are coded with static rANS [10] using a separate frequency table per exponent of the *base* value, which gives 256 contexts per stream. Tables are quantized to 2^{14}, stored in a compressed header, and shared by independent 1M-element chunks, so that encoding and decoding run in parallel. Bases (standalone tensors) are coded as sign, exponent, and mantissa streams, with the mantissa conditioned on the exponent.

Why this context and not more. We measured richer contexts on every stored tensor of the Qwen optimum. We tried the base's top mantissa bits, the neighbouring residuals, whether the high part is zero, and per-chunk adaptive tables. The best of them improves total reduction by 0.03 points. Given the base exponent, residual symbols are close to independent and identically distributed, so further gains must come from better *prediction* rather than richer order-0 contexts.

4.3 Low-rank prediction

Many hub models are LoRA fine-tunes that were merged into the base before upload, so $t \approx b + US \cdot V$ with a small inner dimension r. For them the residual against the parent is structured but not small, since it is a dense rank-r matrix. Tessera stores such a tensor as bf16 factors US ($m \times r$) and V ($r \times n$), plus the base-conditioned residual of t against the prediction $P = \mathrm{bf16}(b + US \cdot V)$.

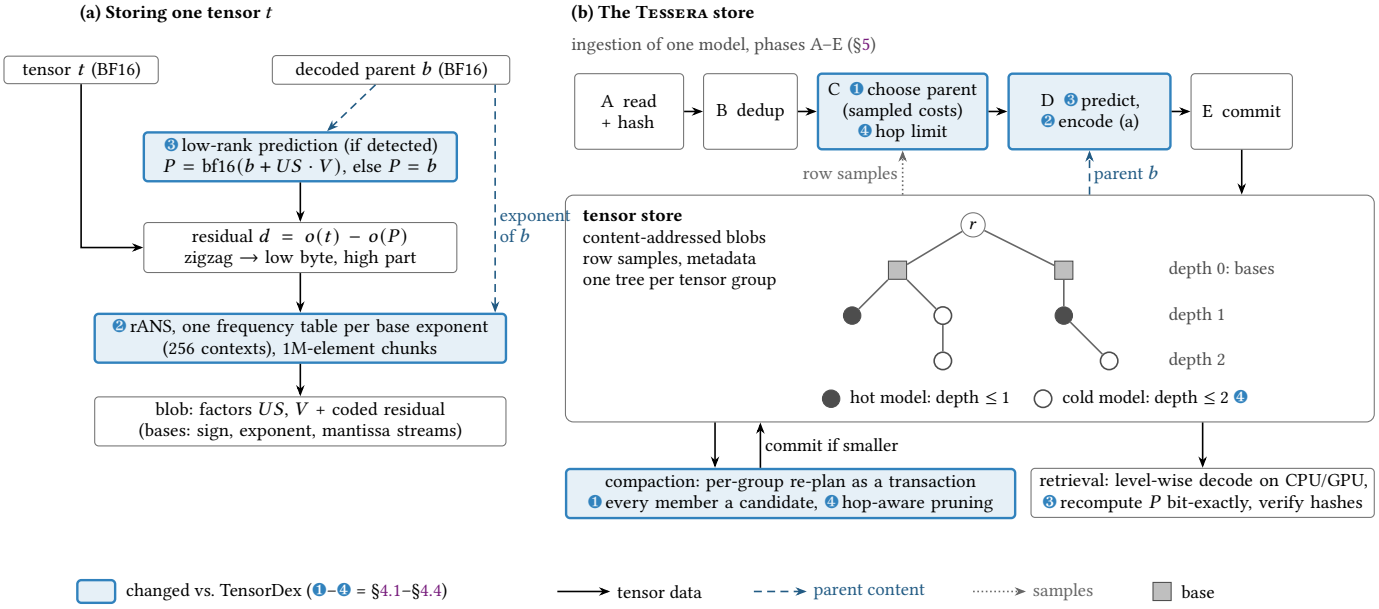


Figure 2: Tessera. (a) Storing one tensor t against its decoded parent b. If a merged low-rank update is detected, t is predicted as $P = \mathrm{bf16}(b + US \cdot V)$ (§4.3); otherwise $P = b$. The order-preserving integer residual of t against P is split into a low byte and a high part, and each is rANS-coded with one frequency table per exponent of the base value (§4.2). Bases are coded as sign, exponent, and mantissa streams. (b) The store. A model is ingested in phases A–E (§5), and each tensor's parent is chosen from sampled costs within its group, subject to its hop limit (§4.1, §4.4). Each group is a tree of blobs rooted at a virtual root r: tensors of hot models sit at depth at most 1 and those of cold models at depth at most 2. Compaction re-plans a group as a transaction and commits only if the group shrinks, and retrieval decodes a model's closure level by level on a CPU or GPU and verifies every tensor. Blue stages ❶–❹ are what Tessera changes relative to TensorDex. Offline plans use the same coding but choose parents with the exact layered ILP.

Detection at low cost. Most tensors are not low-rank, and an SVD per ingested tensor would dominate ingestion time. Tessera first computes the singular values of 256 sampled rows of $t - b$ from their 256×256 Gram matrix. It rejects the tensor if the sampled spectrum is flat ($\sigma_{64}/\sigma_0 > 0.15$). This rejects 46% of the projection tensors on Qwen and loses only 0.05 points of reduction. For the rest, one randomized SVD [14] of rank 64 is followed by a rank choice among 13 candidates, made by estimating each candidate's residual entropy on a row sample. The low-rank encoding is kept only if its estimated gain exceeds 10%.

Bit-reproducible decoding. Losslessness now depends on the decoder recomputing P *bit-exactly* on any machine. BLAS libraries reorder floating-point sums depending on threads and vector width, and compilers may contract multiply-adds into FMAs. Tessera therefore computes P with its own kernel. Every output element accumulates $b + \sum_k u_k v_k$ in fp32 in ascending k, compiled without fast-math, so no FMA contraction or reassociation occurs, subnormals are kept (no flush-to-zero), and the result is rounded to BF16 with round-to-nearest-even. The result is independent of thread count and vector width, and equals a scalar IEEE reference. We verified it across an x86-64 server and an arm64 laptop (§6.7). The GPU kernel performs the same sequence with explicitly non-fused operations and matches it bit for bit (§6.4).

4.4 Popularity-aware hop limits

A one-hop design treats every model as if it were downloaded equally often. In practice, download counts are highly skewed. Tessera assigns each model a hop limit: hot models (by default, the top half by downloads) keep one hop, and cold models may be stored two deltas deep. This is the recreation/storage tradeoff of dataset versioning [7], with the recreation bound set per model by popularity and expressed as chain depth, which the decoder pays in blob reads. A tensor shared by several models after deduplication takes the smallest of their limits. We solve the problem of §2 exactly per group with a layered integer program. Binary z_j stores j standalone, and x^{ℓ}_{ij} places j at depth $\ell \le h_j$ below i:

$$\begin{aligned} &\min \ \textstyle\sum_j a_j z_j + \sum_{\ell,i,j} c(i,j)\, x^{\ell}_{ij} \\ &\text{s.t. } z_j + \textstyle\sum_{\ell,i} x^{\ell}_{ij} = 1 \ \ \forall j, \qquad x^{1}_{ij} \le z_i, \\ &\qquad x^{\ell}_{ij} \le \textstyle\sum_k x^{\ell-1}_{ki} \quad \text{for } 2 \le \ell \le h_j. \end{aligned}$$

It has $O(Hn^2)$ constraints and $O(Hn^3)$ nonzeros for $H = \max_j h_j$. Online, the store offers only parents whose depth leaves room under the tensor's limit. Two hops capture nearly all of the multi-hop gain. Unbounded chains, which reach depth 13 on Qwen, add 0.05 points over two hops but read 1.5× as many bytes (§6.3).

5 Implementation

Tessera is about 3,800 lines of Python (including CUDA kernels compiled at run time) plus 360 lines of C, with 51 test functions. The tests cover round-trips, byte-identity between implementations, decode determinism on CPUs and GPUs, the equivalence of parallel and serial ingestion and compaction, and recovery from a crashed encode worker.

Codec. The rANS coder has three implementations that produce the same byte stream, so each can decode the output of the others. The C implementation, called through ctypes so that the GIL is released, is the production path. It interleaves four rANS lanes per chunk, renormalizes branchlessly, and uses packed decode tables. A numba version serves as a reference. A GPU decoder (CUDA through NVRTC) reads a *strided* layout, in which chunk k holds elements $k, k+C, k+2C, \ldots$ with $C \approx n/4096$, so that one GPU thread per chunk reads the base and writes the output with coalesced accesses. The C library decodes this layout on CPUs as well. Its size overhead is 0.56% for bases and a fixed 36 bytes per 4,096 elements for deltas. Low-rank predictions run in a GPU kernel that reproduces the CPU kernel's operation order with non-fused multiplies and adds (`__fmul_rn`, `__fadd_rn`, compiled with `-fmad=false`). Retrieval batches each dependency level into at most three launches (predictions, deltas, bases) over device-side descriptor arrays.

Store layout. Each stored tensor is one blob named by the content hash of its decoded bytes (blake2b-128, TensorID). A metadata table records every tensor's group, parent, depth, kind (standalone, delta, or low-rank), size, reference count, and children, and every model's name-to-hash map. Row samples for cost estimation live next to the blobs.

Parallel ingestion. Tensor names are unique within a model, so a model adds at most one tensor to each group, and parent candidates come only from earlier models. Every decision for a tensor therefore depends on the store state *before* the model, not on the model's other tensors. Tessera exploits this with five phases per model: (A) read and hash all tensors in parallel; (B) deduplicate and assign sample rows to new groups serially, in file order; (C) choose parents in parallel; (D) fetch parent contents, predict, encode, and write blobs in parallel, largest first; (E) commit metadata serially, in file order. Phase A of the next model runs ahead while the current model is in phases C–E. All heavy work in the parallel phases runs without the GIL, in C called through ctypes, `nogil` numba kernels, hashlib, and numpy copies. BLAS is limited to one thread per call. Parent contents that several tasks need are decoded once, by whichever task needs them first. The result is identical to serial ingestion, with the same parents and byte-identical blobs, which we check in the tests and on real models (§6.4). Size estimates (histogram plus entropy), the low-rank rank search, and the detection Gram matrix are single `nogil` numba calls. Low-rank encodes run in worker processes, which receive tensors through shared memory and call the same encoder, so the blobs are byte-identical.

Compaction and deletion. A tensor's parent always belongs to its own group, so compaction runs one task per group in parallel, and decodes and re-encodes inside a group in parallel as well. Tasks never write shared state other than their group's metadata and blobs, and sampled pair costs are cached across compactions. Planned changes are pruned per tensor with the hop limits of the models that reference each tensor, and groups with a predicted gain below 0.5% are skipped before anything is decoded. Compaction re-plans a group from its samples and applies the new plan as a transaction. Changed tensors are re-encoded next to their old blobs, bases first, and the group is committed only if its total size shrinks; otherwise it is rolled back. A per-tensor "only if smaller" rule would forbid promoting a new base, whose standalone blob is larger than its old delta, even when the group as a whole shrinks. Deleting a model drops its references. Unreferenced leaves are removed immediately, and unreferenced tensors that still parent others are re-parented by the next compaction.

Retrieval. A model's closure, its tensors and their ancestors, is decoded level by level on one thread pool. Blobs are read zero-copy, and decode tables are prepared as separate tasks. Bases start decoding as soon as they are read, low-rank predictions run in row blocks, and every chunk range is its own task. Each tensor is verified, by blake2b the first time and afterwards by the xxh3-128 digest recorded on that first verified retrieval.

6 Evaluation

We answer five questions. How much does Tessera reduce storage, and where do the gains come from (§6.1)? How close are practical planners to the optimum (§6.2)? What do deeper chains cost at read time (§6.3)? How fast are ingestion and retrieval (§6.4)? Does the design hold up online and across machines (§6.5–6.7)?

Setup. All CPU experiments run on a shared server with a 208-thread Intel Xeon Platinum 8555C and 1 TB of DRAM; models are staged on a local md-RAID disk. GPU experiments use one NVIDIA H200. The corpus is described in §3.1. Unless marked otherwise, every storage number comes from real encodes, in which each tensor is encoded as planned, decoded root-first along its chain (children against their parent's decoded bytes), and compared to its content hash. Baselines run the authors' code and are round-trip checked too. The exceptions are DFloat11 (computed payloads) and Figures 3 and 6 and Table 7 (exact planned sizes of Tessera's plans, within 0.04 points of the verified ones). Throughput is not comparable to TensorDex's numbers, which were measured on a dedicated 192-thread AMD machine with all data in DRAM.

6.1 Storage reduction

Table 3 and Figure 4 show the progression. Pooled over the four families, Tessera stores 206.3 GB of the 816.9 GB. Our re-implementation of the TensorDex design stores 389.2 GB, and the best pairing-only design, the exact one-hop optimum with the XOR/zstd codec, stores 331.2 GB. With Tessera's hop limits, pairing alone reaches 60.3%, and with chains of any length 60.6%, 14 points below Tessera.

Baselines. Single-model coders save a third in every family. ZipNN and DFloat11's Huffman-coded exponent agree to within 0.05 points, and Tessera's codec without a base adds under one point, so exponent coding is saturated and further savings must come from other models. FM-Delta, run with the authors' code (BF16 through its 16-bit integer path, every tensor round-trip checked) against each family's pretrained model, reaches 46.4–56.2%, 6–15 points below the one-hop optimum, because the pretrained model is the best base for only part of each family. Given

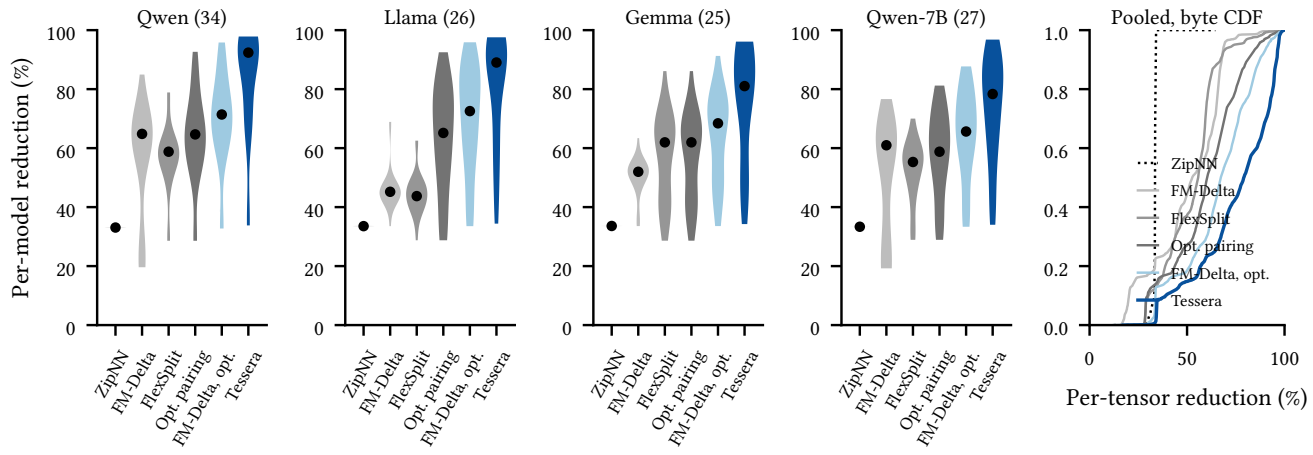


Figure 3: Per-model reduction per family (violins; dots are medians) and per-tensor reduction weighted by bytes, pooled. "Opt. pairing" is the exact one-hop optimum under XOR/zstd; "FM-Delta, opt." codes the same pairs with FM-Delta. Models are those that introduce new tensors.

Table 3: Reduction (%) after deduplication, per family and pooled over the four families. "One-hop optimum" is the exact optimum of base selection for the given codec. †Computed payload (Huffman-coded exponent, lookup tables excluded); every other row is encoded and verified.

	Qwen	Llama	Gemma	Qwen-7B	Pooled
Deduplicated raw (GB)	187.5	145.8	108.9	374.8	816.9
zstd, raw bytes	21.5	22.3	21.7	22.1	21.9
ZipNN	33.1	33.6	33.6	33.3	33.4
DFloat11†	33.1	33.6	33.6	33.3	33.4
Tessera codec, no base	33.9	34.4	34.3	34.1	34.1
FM-Delta, pretrained base	56.2	46.4	49.9	48.9	50.3
FM-Delta, optimal pairs	69.1	68.2	62.6	64.0	65.7
ZipLLM, declared bases	66.0	60.9	60.8	64.4	63.7
FlexSplit, XOR/zstd (our re-impl.)	56.5	44.7	56.2	52.2	52.4
FlexSplit pairs, Tessera codec	67.3	56.1	67.5	62.5	63.1
+ low-rank prediction	69.6	56.8	71.6	63.0	64.5
FlexSplit, all candidates	62.8	61.4	56.2	56.8	58.9
Model-level one-hop optimum	62.4	61.2	55.2	57.1	58.8
Tensor-level one-hop optimum	63.0	61.6	56.2	57.8	59.5
same, hot 1-hop / cold 2-hop	64.5	62.3	56.2	58.6	60.3
+ base-conditioned codec (opt.)	73.1	72.9	67.6	67.8	69.9
+ low-rank prediction	79.7	76.4	72.2	70.3	73.8
+ hot 1-hop / cold 2-hop (Tessera)	**80.9**	**77.3**	**72.2**	**71.4**	**74.7**

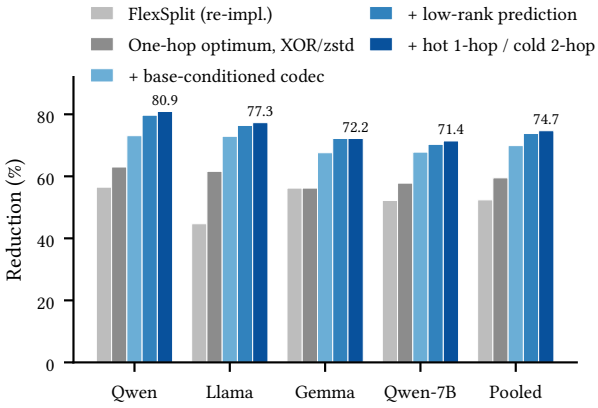


Figure 4: Contribution of each technique per family (reduction after deduplication).

the optimal pairs, FM-Delta's range coder over integer deltas gains 6.1–6.6 points over XOR/zstd, close to the 5.4 points TensorDex reports for FM++ over TensorX. On the same Qwen pairs Tessera's codec reaches 73.0%, 3.9 points more (4.7, 4.9, and 3.8 points on Llama, Gemma, and Qwen-7B after re-planning). FM-Delta models the magnitude class of every delta with one adaptive distribution per tensor, whereas Tessera conditions it on the base exponent. FM-Delta's coder also runs at 52–77 MB/s per thread. ZipLLM, run with the authors' code at a pinned commit, deltas each fine-tune against its declared base with BitX, which splits the XOR into byte planes before zstd, and reaches 60.8–66.0% (63.7% pooled) with byte-exact restores, counting its compressed data and metadata but not the raw copies its prototype also keeps. On Qwen the same declared pairs give 62.4% with plain XOR/zstd (Table 2), so again the codec, not the pairing, makes the difference. Tessera stores 30% fewer bytes than ZipLLM (206.3 vs 297.0 GB).

Distribution. The gains are not carried by a few models (Figure 3). The median fine-tune is reduced by 92.4, 89.0, 81.0, and 78.3% (Qwen, Llama, Gemma, Qwen-7B), versus 64.7, 65.1, 62.0, and 58.8% under the optimal XOR/zstd pairing. The lower tail consists of the roots of each sub-family (the Instruct and base models, stored standalone at 34% and serving as bases) and of heavily retrained derivatives. Per tensor, half of all bytes belong to tensors that Tessera reduces by more than 79%, and 31% belong to tensors

reduced by more than 90%, against 3% under the optimal pairing. Of the 130 models, 18 introduce no new tensor after deduplication and are not shown.

The gains over the pairing-only optimum decompose as follows.

Codec: +10.0 to +11.4 points in every family. This is the most consistent gain. On Qwen, 3.9 points come from replacing zstd, a poor entropy coder for these symbol streams (the order-0 entropy of the same residuals already reaches 66.9%). The other 6.2 points come from conditioning on the base exponent. On Qwen, all 11,117 stored tensors are encoded and hash-verified at 73.03%, which matches the entropy estimate we used to screen the idea (73.03%).

Low-rank: +2.5 to +6.6 points, tracking LoRA prevalence. On Qwen, 13 of the 34 models that add new tensors are LoRA merges of rank 4–32, and their delta bytes shrink by 57–86%. In the default plan, low-rank prediction codes 2,939 of the 7,986 distinct projection tensors. Llama (+3.5) and Qwen-7B (+2.5 points) gain least, because it applies to fewer of their tensors (989 and 722 projections). Low-rank prediction also needs the right base. On the pairs of our FlexSplit it adds only 1.4 points pooled (0.6 on Llama, 0.5 on Qwen-7B), because the LoRA source is often not the base FlexSplit chose. On the pairs of the all-candidates variant it adds 3.7 points (69.2 to 72.9%). The choice of base thus matters more once residuals are predicted.

Hops: 0 to +1.2 points. The gain depends on lineage depth. Gemma is flat, so a second hop helps only when fewer models are hot (+0.9 points with the top 20% hot). On top of low-rank prediction the gain shrinks slightly (Qwen: +1.2 points versus +1.6 without low-rank), because merged LoRA deltas are already small.

What does not help. A synthetic bitwise-majority base never beats the real base (0 of 1,075 Qwen clusters), a least-squares mix of two bases gains 0.16 points, and richer entropy contexts at most 0.03.

6.2 Planner quality

Figure 5 tracks planners as the Qwen corpus grows. Our FlexSplit falls behind early, once the Instruct sub-family forms, and never recovers. The all-candidates variant tracks the tensor-level optimum throughout. Model-level choices, whether declared metadata or the model-level optimum, are within 0.6 points. The same ordering holds with the codec of §4.2 on Qwen, where our FlexSplit reaches 67.5%, the all-candidates variant 72.8%, and the optimum 73.1%. The sample-based cost predictor ranks candidate bases better than TensorSketch+TensorPred in the three families where we ran it, with a 10–12× lower mean regret. Its absolute estimates are biased low, which does not affect the ranking and could be removed by an affine calibration. It samples 1.3–5.3 GB to score all 434×1,600 Qwen pairs, and runs in 74–78 s.

6.3 Storage versus read cost

Downloads are highly concentrated. The top 8 of 40 Qwen models receive 99.7% of them, and Qwen2.5-3B-Instruct alone about 80%. Keeping the top half one-hop and letting the rest chain twice is a Pareto improvement over the one-hop design (Table 4, Figure 6). It stores 1.2 points more at nearly the same download-weighted

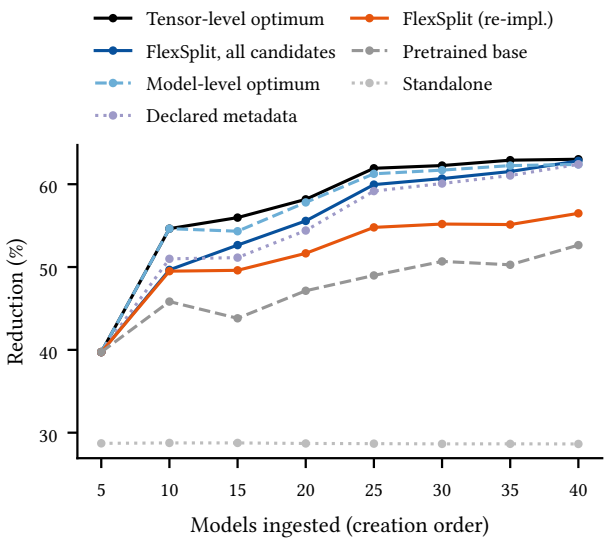


Figure 5: Cumulative reduction as Qwen models arrive in creation order (XOR/zstd codec).

Table 4: Hop configurations on Qwen (codec + low-rank; verified). Read amplification and blobs per tensor are weighted by downloads.

Configuration	Reduction	Read amp.	Blobs/tensor
Hot models pinned as bases	78.70%*	0.663	1.00
All one-hop (TensorDex's space)	79.70%	0.683	1.28
Top 50% one-hop, rest two-hop	**80.89%**	**0.686**	**1.25**
Top 20% one-hop, rest two-hop	81.07%*	0.716	1.39
All two-hop	81.04%	0.745	1.55
Unbounded	81.09%*	1.096	3.93

*planned (exact) size; the other rows are encoded and verified.

read cost (0.686 vs. 0.683 of the raw bytes) and with fewer blobs per downloaded tensor. Unbounded chains add only 0.05 points beyond two hops (81.09% vs. 81.04%) but read 1.5× as many bytes. A storage-only objective can demote a hot model from base to delta. If latency matters most, pinning the hottest models as bases gives one blob per hot download for 1 point of storage.

Comparison with dataset-versioning algorithms. Hop limits trade storage for recreation cost, the problem studied for dataset versioning [7]. We run its heuristics on the same costs (Figure 6). LMG starts from the minimum-storage arborescence (MCA) and repeatedly materializes the version with the best ratio of recreation cost saved to storage added [7]. LMG-All also considers re-parenting moves [13]. We give both the download weights, so they optimize the weighted read cost on the x-axis. MP (modified Prim's) minimizes storage under a bound on every version's recreation cost [7], and we sweep the bound. At the read cost of Tessera's default plan, the best LMG or LMG-All plan stores 2.4, 3.7, 0.9, and 1.1 points less (Qwen, Llama, Gemma, Qwen-7B), with chains 5 to 11 deep instead of 2. Even Tessera's one-hop plan stores less. Bounding the worst model instead, MP and LMG-All reach at most 78.2% on

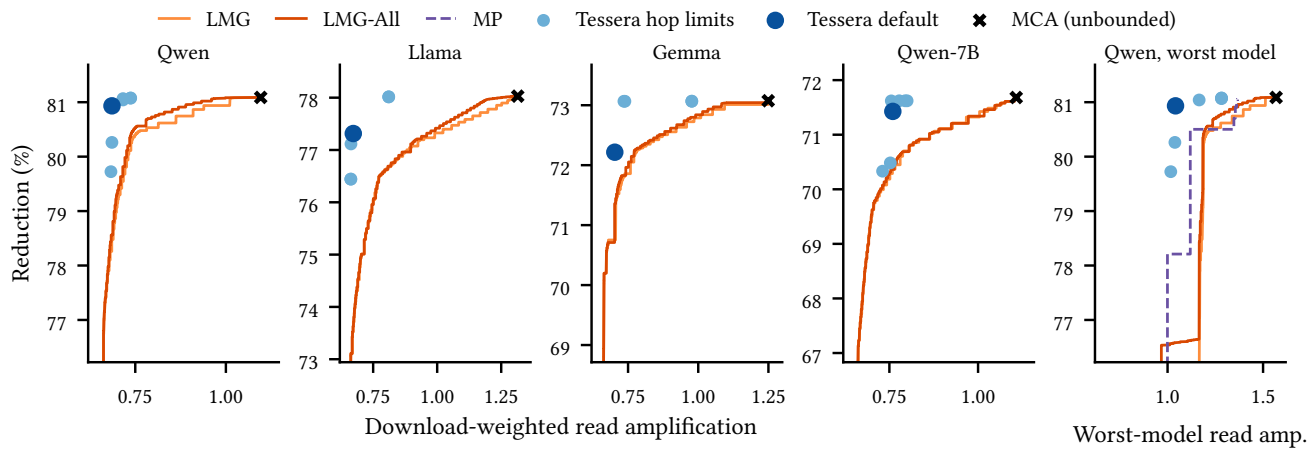


Figure 6: Storage versus recreation cost on codec + low-rank costs: Tessera's exact hop-bounded plans against dataset-versioning heuristics (lines are each heuristic's best plan at or below a read cost). Read amplification is the bytes read to recreate a model over its raw size, weighted by downloads (left four) or for the worst model (right).

Qwen at Tessera's worst-model read cost (1.04), versus 80.9%. The heuristics start from a deep minimum-storage tree and pay a full standalone tensor for each chain they shorten, whereas the exact hop-bounded plan chooses all bases of a group jointly. Depth also matters beyond bytes: every hop is a dependent decode step, and the GPU decoder processes chain levels one after another (§5).

6.4 Throughput

Table 5 summarizes the speed of every stage.

Codec kernels. On a 720 MB tensor with 128 threads, the C codec decodes deltas at 7.4 GB/s and bases at 9.2 GB/s (0.19 GB/s per thread; 0.60 GB/s on an Apple M-series core). Context-based rANS does more work per element than byte-plane Huffman or zstd, which is the cost of the extra 10 points. In one harness (a 6.17 GB fine-tune and its parent in DRAM, 16 MiB chunks dealt to 128 single-threaded processes, every chunk round-trip checked), Tessera decodes deltas at 13.4 GB/s, 1.8× FM-Delta but a quarter of ZipNN and zstd, which use no base. Encoding is Tessera's weakest stage (3.1 GB/s against 9.4 GB/s for FM-Delta), but in ingestion low-rank fitting, not encoding, is the bottleneck.

Ingestion. Parallel ingestion produces the same parents and byte-identical blobs as serial ingestion, and every full run ends at the serial store size. The 40 Qwen models ingest in 133–147 s of wall-clock time over five runs (median 136 s, 1.81 GB/s) instead of 1,993 s serially, a 14× speed-up. Reading and hashing the next model overlaps the current model's encoding. The main obstacle was the GIL (a low-rank encode takes 0.47 s alone but 1.49 s next to 63 threads), so Tessera runs rank search and entropy estimates as single `nogil` numba calls and low-rank encodes in worker processes over shared memory. This doubles whole-corpus ingestion (0.79–0.92 → 1.68–1.86 GB/s), though not on the first eight models, which hold few low-rank tensors (Figure 7). The remaining bottleneck is low-rank fitting, which a smaller randomized SVD did not speed up

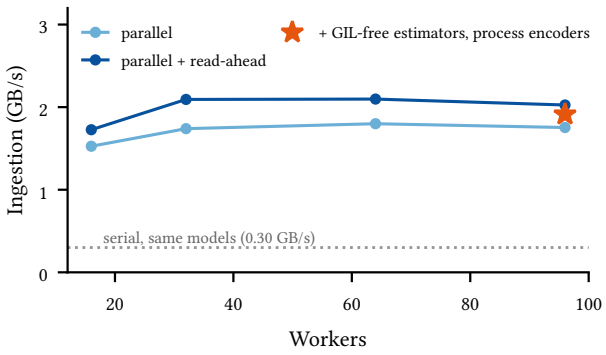


Figure 7: Ingestion throughput (wall clock) on the first eight Qwen models.

(it cost 0.21 GB). All rates are wall clock, and summing per-model times would overstate them by about 30%.

Compaction. Groups are independent transactions, so Tessera compacts them in parallel, and without the attempt filter described below the result is identical to serial compaction. Three measures take the four compaction rounds of the online run from 2,581 s to 47–54 s (Table 5). (i) *Parallelism across and within groups.* With GIL-free estimators, and with decodes and re-encodes inside each group also running in parallel (level by level along the chains), the rounds take 23, 14, 23, and 32 s, and the workers are 98–99% busy. (ii) *Cached pair costs.* Samples never change, so sampled costs from earlier rounds are reused and only pairs involving new members are estimated. (iii) *Avoiding rollbacks.* Per-group records show that 791 of 1,497 group transactions were rolled back and took 54% of the compaction time. The re-plan is one-hop, so it also flattens two-hop chains, which the sampled costs predict to be worse and the real

Table 5: Throughput on the Qwen corpus (40 models, 6.17 GB each). CPU rows: shared 208-thread Xeon 8555C, local disk; GPU rows: one H200. All rates are wall clock and every retrieval is verified. Ingestion and compaction: range of five runs; codecs and retrieval: medians of five runs (load average 55–80 from other users during the CPU retrieval runs). TensorDex's reported numbers come from a dedicated 192-thread AMD machine with data in DRAM and are shown for scale only.

Operation	Configuration	Throughput / time
Ingestion (40 models)	serial	0.12 GB/s (1,993 s)
	parallel + read-ahead, threads	0.79–0.92 GB/s (267–314 s); first 8 models 2.1 GB/s
	+ GIL-free estimators, low-rank encodes in processes	1.68–1.86 GB/s (133–147 s; median 136 s)
Compaction (4 rounds)	serial	2,581 s
	parallel groups, 64 workers / + cost cache	253 s / 198 s
	+ GIL-free estimators, in-group parallelism	91 s
	+ hop-aware pruning, gain filter (80.68% instead of 80.53%)	47–54 s
Codec kernels (CPU)	delta encode, 128 threads	4.0 GB/s
	delta / base decode, 128 threads; delta decode, 1 thread	7.4 / 9.2 GB/s; 0.19 GB/s
Codecs, one harness (one model in DRAM, 128 processes)	zstd, raw bytes (encode / decode)	24.7 / 53.4 GB/s
	ZipNN, no base	18.2 / 59.4 GB/s
	FM-Delta, against the parent	9.4 / 7.5 GB/s
	Tessera delta; Tessera standalone	3.1 / 13.4 GB/s; 3.0 / 12.7 GB/s
Retrieval, CPU	96 workers, cold local disk	1.7–4.0 GB/s (1.55–3.72 s per model)
	4 / 16 concurrent clients, 48 requests	4.5 / 3.8 GB/s total; median 4.8 / 20.9 s
Retrieval, GPU	whole closure incl. low-rank, batched launches	52–120 GB/s (0.05–0.12 s per model)
	+ upload of compressed blobs from pinned memory	0.13–0.20 s per model
Uncompressed pull	local disk / network file system	3.8 / 8.3 GB/s
TensorDex (reported)	TensorX compress / decompress	22.9 / 28.4 GB/s

encode confirms. Skipping whole groups with a small predicted gain removes the waste but costs 0.51 points of reduction (80.02%). Beneficial changes are dropped together with harmful ones, and later models then attach to worse bases. Tessera therefore prunes per tensor first. A tensor keeps its current parent when that parent is predicted at least as cheap as the planned one, the tensor is not itself a planned parent, and all hop limits still hold. Only after that does Tessera skip groups whose predicted gain is below 0.5%. Pruning alone improves storage, from 80.53% to 80.68%, but keeps most rollbacks (76 s). Adding the filter brings compaction to 47 s in the run shown (23, 3, 4, and 18 s; 55× serial) with 21 rollbacks at the same 80.68%.

Retrieval on CPUs. Reconstructing a 6.17 GB Qwen model from a cold local disk, with every tensor verified, takes 1.55 s for a model stored mostly as bases (4.0 GB/s), 1.92 s for a LoRA-merged model, and 3.72 s for a model with many low-rank tensors (medians of five runs; range at most 0.4 s except one 2.6 s outlier). Reading the model's compressed closure (4.1–4.4 GB) takes 0.7 s, so retrieval is CPU-bound. On local disk Tessera matches pulling the uncompressed checkpoint (3.8 GB/s) for base-heavy models while reading 28–34% fewer bytes, but low-rank-heavy models take up to twice as long, and on the faster network file system (8.3 GB/s uncompressed) Tessera is slower. Verification costs 0.1 s with recorded xxh3 digests, versus 1.2 s with blake2b. Under concurrent load, with 48 requests drawn by download count and split over K clients, the aggregate rate saturates at 4.5 GB/s (K=4; 3.8 GB/s at K=16), so median latency grows almost linearly with K (1.4, 4.8, 10.1, and 20.9 s for $K = 1, 4, 8, 16$; p99 4.1 to 29.2 s). A CPU-only deployment should therefore serve its most popular models from a decoded cache (§7).

Retrieval on a GPU. The GPU decoder handles the whole closure of a model, including two-hop chains and low-rank tensors. The prediction kernel accumulates in the same order as the CPU kernel with explicitly non-fused IEEE operations, so it matches the CPU bit for bit (tested on shapes of up to 65,600 rows). With one launch per tensor, whole-model decoding (0.10–0.22 s for the three models above) is bounded by the many small launches, while the kernels alone reach 114–131 GB/s on large tensors. Tessera therefore batches each dependency level into at most three launches (predictions, deltas, bases) over descriptor arrays, and every thread finds its tensor by binary search over a prefix sum of chunk counts. One H200 then decodes the three models in 0.051, 0.062, and 0.118 s (120, 100, and 52 GB/s; medians of five runs), bit-exact, and in 0.13–0.20 s including the upload of the 4.1–4.5 GB of compressed blobs from pinned memory. Tiny tensors (229 per closure) are decoded on the CPU concurrently. The GPU layout costs 0.6–0.9% of the stored bytes.

6.5 Online operation across families

The offline results assume that every model is known in advance and that bases are chosen from exact encoded sizes. A hub instead receives models one at a time. We therefore replayed each family through the store in creation order, using sampled-cost parent choice with popularity-based hop limits, the low-rank path, and parallel compaction every 10 models. Each run ingests exactly the tensor set of the offline experiments, so the deduplicated totals are the same (187.5, 145.8, and 108.9 GB). Every model is retrieved and hash-verified at the end.

Table 6: Online replay of each family in creation order, against the offline optimum with the same techniques (codec, low-rank, top 50% one-hop / rest two-hop). Ingestion is wall clock; Qwen was read from local disk, Llama and Gemma from a shared network file system that bounds their ingestion.

	Qwen	Llama	Gemma
Offline one-hop optimum	79.70%	76.43%	72.20%
Offline optimum (hot 1, cold 2 hops)	80.89%	77.30%	72.20%
Online	**80.68%**	**75.86%**	**71.45%**
Gap to the offline optimum (pts)	0.20	1.44	0.75
Ingestion (wall)	147 s	457 s†	439 s†
Compaction (all rounds)	52 s	65 s	11 s
Rollbacks / attempted groups	21 / 698	0 / 413	1 / 190
GPU retrieval (top 3 models)	0.05–0.09 s	0.05 s	0.04–0.07 s

† of which reading from the network file system: 426 s and 422 s.

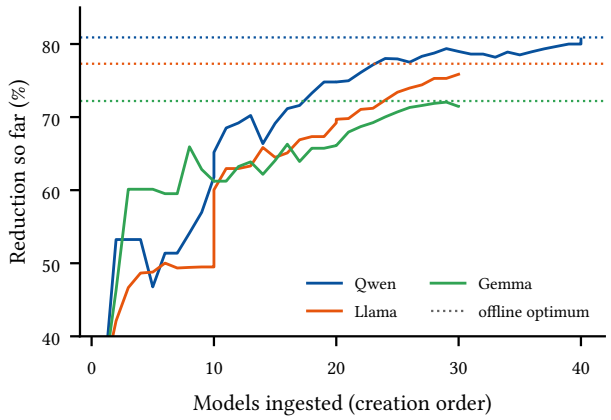


Figure 8: Reduction of the online store as models arrive (compactions after every 10 models show as jumps); dotted: offline optimum with the same techniques.

Online stays close to the offline optimum. The online stores end 0.20 (Qwen), 1.44 (Llama), and 0.75 points (Gemma) below the offline optimum (Table 6). On Qwen the online store beats the offline *one-hop* optimum, because ingestion attaches cold models two hops deep. On Llama and Gemma it does not.

Compaction repairs the early choices. A model can only attach to models that arrived before it, so the first models are placed poorly. In Llama, the pretrained model arrives first, and 10 models later the store is at 49.5%. The first compaction lifts it to 60.0% (Figure 8). On Qwen the first round adds 3.4 points, and the flat Gemma family has nothing to repair. Later rounds add at most 0.7 points. Rollbacks are rare in every family (0–3% of the attempted groups).

Where the remaining gap comes from. The online layout is already optimal *in its own cost model*. On the final Llama store, compaction's local search, a fresh local search, and the exact one-hop ILP all reach the same predicted size. The gap comes from the compaction schedule (below) and from the cost model, which estimates *plain* deltas while the offline optimum plans with real encoded sizes that include low-rank coding. The online stores therefore code fewer tensors low-rank (816 vs. 989 in Llama, 1,016 vs. 1,215 in Gemma). Probing every candidate for its LoRA source gained only 0.01–0.03 points and slowed Qwen's ingestion by 27%, so it is off.

Mixing families. Ingesting the 100 models of the three small families into one store, in creation order, loses nothing. No group spans two families, and the store ends at 77.26% against 76.82% for three separate stores, every model hash-verified. Llama gains 1.3 points (77.14%, 0.16 below its offline optimum), because its groups, interleaved with the other families, are re-planned every three Llama arrivals instead of every ten, and each re-plan retries low-rank coding against the new parents.

A random, heterogeneous corpus. To test our selection bias, we sampled uniformly at random (seed 0), regardless of downloads or organization, from the 13,432 hub fine-tunes of eight base models in four series (Qwen2.5 0.5/1.5/3B, Llama-3.2 1/3B, Gemma-2 2B, SmolLM2 0.36/1.7B), keeping ungated pure-BF16 repositories, and added the 13 declared bases missing from the sample, as a hub stores them. Dropping one repository that holds two conflicting checkpoints leaves 112 models (464 GB, 392 GB after deduplication; median fine-tune: 10 downloads). Ingested online in creation order, Tessera stores 92.7 GB, a 76.4% reduction, with every model hash-verified. ZipNN reaches 33.4%, zstd 21.8%, and ZipLLM 65.1% (Tessera stores 32% fewer bytes).

Speed and deletion. The whole online Qwen run, 40 models and four compactions, takes 186–200 s in five runs instead of 3,547 s serially. From the online stores, one H200 reconstructs the three most-downloaded models of each family in 0.035–0.088 s each, bit-exact. Deleting the five least-downloaded Qwen models (in an earlier run, 80.53%) freed 2.01 GB at once and 2.55 GB after the next compaction, and all remaining models verified.

6.6 Sensitivity and overheads

Table 7 varies Tessera's parameters on Qwen, re-planning exactly for every setting, and the defaults sit on a plateau. Two low-rank candidate parents already reach the default and five gain nothing, while one parent loses 0.7 points. Disabling the flatness filter gains 0.05 points for 1.7× the low-rank encoding time. Seven power-of-two ranks do as well as thirteen, but any single fixed rank costs 37–47% more low-rank bytes. Chunk size and table precision change the stored size by under 0.5% (16-bit tables would overflow the decoder's packed fields and are rejected). Cost samples beyond 64 K elements per tensor no longer improve base choice.

Planner cost. On the real groups (at most 34 unique tensors each), every exact program was solved to proven optimality, in at most 0.17 s (one hop) and 0.34 s (hop-bounded) per group; all 976 groups take 29 s and 88 s of CPU time, and the store's local search matches the one-hop optimum. On groups bootstrapped from the largest Qwen projection group, the one-hop program needs 22 s at n=320 versions but does not prove optimality within 30 minutes at 640 (Edmonds: 11 minutes), and at 1,280 building it exceeded 200 GB. Local search stays within 0.3% of the optimum at 320 (372 s). Hub families already reach thousands of fine-tunes, so such groups need decomposition, for example by sub-family.

Table 7: Sensitivity on Qwen (exact re-planning per setting; planned sizes; defaults in bold). Low-rank encode time is CPU time over all candidate edges; rank-set costs are measured on 164 targets.

Parameter	Settings	Result
Low-rank candidate parents K	0 / 1 / 2 / **3** / 5	74.67 / 80.26 / 80.93 / **80.93** / 80.93%
Flatness filter τ (σ_{64}/σ_0)	0.05 / 0.10 / **0.15** / 0.30 / off	80.56 / 80.87 / **80.93** / 80.97 / 80.98%; encode 5.0 / 6.5 / **7.1** / 8.5 / 12.3 k CPU-s
Candidate ranks	**13** / 7 (powers of 2) / 4 / {16} / {64}	low-rank bytes +0 / +0.25 / +1.5 / +37 / +47%
Hot fraction (one-hop)	10 / 20 / 30 / **50** / 90% of models	81.08 / 81.07 / 81.04 / **80.93** / 80.26% at read amp. 0.737 / 0.716 / 0.716 / **0.686** / 0.686
Codec chunk; table precision	2^{16}–2^{22} (2^{20}); 12–15 bits (**14**)	stored size within 0.5%; all round trips exact
Cost sample per tensor	16 K / 64 K / 256 K / 1 M elements	regret 0.063 / 0.055 / 0.055 / 0.054 points (TensorPred: 0.50)

Metadata. The store's metadata takes about 440 B per tensor as JSON, under 0.02% of the stored bytes (64 B in a fixed binary layout). The online planner's cost-sample cache is larger, at 0.8–1.4% of the stored bytes. Counting it lowers the online reductions of Table 6 by 0.20–0.27 points (Qwen 80.41%, Llama 75.66%, Gemma 71.21%). Low-rank factors make up 0.7–2.7% of the stored bytes.

6.7 Cross-machine determinism

We encoded the projections of four LoRA-merged Qwen models on the x86-64 server and decoded them on the server and on an arm64 laptop. All low-rank predictions were bit-identical to a scalar IEEE reference, with one thread and with all threads, and all decodes matched their hashes. The machine code of a freshly compiled prediction kernel contains no FMA instruction on either architecture; compiled with fast-math, it contains 14 (x86) and 6 (arm64). The test suite checks both properties on every machine it runs on.

7 Discussion and Limitations

Scope of the corpus. Our ground truth covers three families of 2–3B-parameter models and one 7B family, with 30–40 popular fine-tunes each, chosen so that exhaustive all-pairs costs are affordable. TensorDex's corpus is 2,890 randomly sampled models from eight families. A random sample of 112 models from eight bases (§6.5) reaches a similar reduction online (76.4%), but has no exhaustive optimum. Two of our conclusions depend on within-family structure. Tensor granularity may matter more in heterogeneous corpora, and FlexSplit's gap varies with family structure, from 0 to 16.9 points in our four families. The codec and low-rank gains, in contrast, are per-pair effects, and they held in every family we measured.

When Tessera does not help. A model without relatives in the store, such as a new base model or a fine-tune that changes the vocabulary or a tensor's shape, forms groups of its own and gains only the third that single-model coding saves. Low-rank prediction pays off only for merged adapters, and hop limits only when downloads are skewed.

Comparison with TensorDex. Our FlexSplit is a re-implementation with three parameters chosen by us (§3.1), and we did not re-implement FM++, reported to add 5.4 points over TensorX. Its predecessor FM-Delta, run with the authors' code on the optimal pairs, adds 6.1–6.6 points over XOR/zstd, and our codec another 3.8–4.9 (§6.1). The one-hop optimum under XOR/zstd bounds every planner with that codec, so our main comparison does not depend on how well we tuned FlexSplit.

Online planning. The online cost model sees only plain deltas. Re-planning more often closes most of Llama's gap (§6.5), and low-rank-aware estimates, for example from a per-tensor spectral sketch, are the natural next step for Gemma's.

Popularity is a snapshot. Hop limits use download counts at selection time. Popularity drifts, so hot models must be re-evaluated periodically, for example when compaction re-plans a group. Pinning the hottest models as bases bounds the worst case for the most frequent downloads.

Throughput and cost. Context-based rANS spends more CPU per byte than byte-oriented coders. Tessera encodes 3–6× slower than FM-Delta and ZipNN at 128 processes, and its CPU decoder does not beat a fast network file system. Ingestion is bounded by low-rank fitting. At list prices (S3 Standard, $0.023 per GB-month [5]; c6a.48xlarge on demand, $0.038 per vCPU-hour [4]), ingesting and compacting the Qwen corpus costs 10.9 vCPU-hours ($0.42, charging all 208 threads for 188 s), and the 45.3 GB that its online store saves over FlexSplit (Table 3) are worth $1.04 per month, so the compute pays for itself in 12 days. Retrieval is different. A CPU decode of 1.55–3.72 s on 96 workers costs as much as storing a decoded 6.17 GB copy for a month after 37–90 downloads. All 40 Qwen models exceed that, while the uniformly sampled fine-tunes of §6.5 have a median of 10. Popular models should therefore be served from a decoded cache or a CDN, and Tessera pays off for the long tail. The GPU decoder (0.13–0.20 s per model including the upload) cuts the decode cost per download by 3–7× even at $10 per GPU-hour. A SIMD CPU decoder is future work.

Quantized derivatives. Hubs also store GGUF quantizations [1] next to the BF16 parent. Predicting their codes from the re-quantized parent, with the AWQ input scaling [30] of the official Qwen2.5-3B files modelled, gives an entropy estimate of 28–32% of their size (zstd: 94–99%). A real encoder is future work.

Determinism as a correctness condition. Any predictive path that the decoder recomputes in floating point is part of the lossless contract. We made the low-rank kernel deterministic by construction and test it on every machine. So far this covers x86-64 and arm64 CPUs and one GPU model (H200). A production system should also pin the kernel version in the blob format.

8 Related Work

Versioned data and model management. Storing many versions as deltas of one another is a classic data-management problem. Bhattacherjee et al. formalize the recreation/storage trade-off: which versions to materialize and which to store as deltas, under bounds on total or maximum recreation cost [7]. Guo et

al. give approximation algorithms for its variants [13], and Decibel and OrpheusDB bring branching and versioning to relational data [22, 33]. For models, ModelHub [34] archives the parameters of deep-learning model versions with delta encoding and the same tradeoff, ModelDB [40] manages model metadata and lineage, MISTIQUE [39] stores model intermediates compactly with quantization and deduplication, and MGit [15] records a lineage graph of derivative models and reduces their storage along it. Tessera's hop limits are a popularity-weighted instance of the recreation/storage tradeoff at tensor granularity, and on our costs its exact plans store less than LMG, LMG-All, and MP (§6.3). These systems take the size of a delta as a given edge weight and optimize over the version graph. For fine-tuned weights, predicting and coding the residual changes the edge weights by more than choosing among them.

Lossless model storage. ZipNN [17] and DFloat11 [44] entropy-code the exponents of individual models and save about 30%. DFloat11 also decodes on the GPU during inference. lmz [2] splits floats into bit fields and uses conditional rANS within one model, and Heilper and Singer extend exponent/mantissa coding to FP8 and FP4 [16]. FM-Delta [35] codes integer deltas of fine-tunes against their pretrained model. ZipLLM [42] combines tensor-level deduplication with XOR deltas (BitX) inside model families found from metadata or bit distance. TensorDex [26] makes base selection tensor-centric with sketches, a learned cost model, and online clustering. Tessera adopts the tensor-group structure and shows that, once pairing is near-optimal, the residual's prediction and coding matter more. Hugging Face's Xet backend [24] deduplicates 64 KB chunks across repositories, which removes exact duplicates but not near-duplicates.

Floating-point compression in databases. Time-series databases compress floating-point streams losslessly: by XOR with the previous value [29, 36], by erasing insignificant mantissa bits [27], or by decimal encoding [3]; Hishida et al. compare such schemes broadly [18]. They exploit redundancy between neighbouring values of one series. Within a weight tensor that redundancy is limited to a skewed exponent distribution (the one third that single-model coders save). The larger redundancy lies across models, where Tessera predicts and codes each value from its base.

Checkpoint compression. Training systems compress consecutive checkpoints losslessly by XOR and byte grouping [41], or by prediction and context modeling from the previous checkpoint [25]. ExCP [28] prunes and quantizes residuals of weights and optimizer state, so it is lossy. These systems code against a fixed predecessor, whereas Tessera chooses bases across models and uses the base as coding context.

Lossy delta compression for serving. BitDelta [31], DeltaCoMe [37], and DeltaZip [43] quantize or sparsify fine-tuning deltas so that many variants can be served from one base. They show that deltas are highly structured (sign-dominated, low-rank, heavy-tailed). Tessera exploits the low-rank structure but stays lossless, as a hub must.

Model lineage. MoTHer [20] recovers model trees from weights alone. The Model Atlas [19] charts the hub and finds that merges, quantizations, and adapters are common. Our results confirm that declared lineage is unreliable. They also suggest that a storage system does not need the true lineage, only good bases, which sampled costs find directly.

Algorithms. Base selection is facility location, for which local search with add, drop, and swap moves [6] and exact integer programming are standard tools. We solve our programs with HiGHS [23] through SciPy. Unbounded multi-hop storage is a minimum spanning arborescence [11]. Tessera uses rANS [10] and randomized SVD [14].

9 Conclusion

Exhaustive ground truth shows that choosing which tensor to delta-encode against is easy to solve near-optimally, and that the larger opportunity lies in the residual. Conditioning the entropy coder on the base, predicting LoRA-merged fine-tunes with a bit-reproducible low-rank kernel, and letting cold models chain twice raise lossless reduction to 74.7% in verified offline plans, against 52.4% for our re-implementation of the TensorDex design, 60.6% for the best pairing, and 63.7% for ZipLLM. Tessera's online store stays within 0.2–1.4 points of those plans and reaches 76.4% on a random sample of the hub.

References

[1] [n. d.]. GGUF file format (ggml / llama.cpp). https://github.com/ggml-org/ggml/blob/master/docs/gguf.md. Accessed 2026-09-25.

[2] [n. d.]. lmz: Fast lossless compression for large model weights. https://github.com/FanxinSun/lmz. Accessed 2026-09-24.

[3] Azim Afroozeh, Leonardo X. Kuffo, and Peter Boncz. 2023. ALP: Adaptive Lossless floating-Point Compression. *Proceedings of the ACM on Management of Data* 1, 4, Article 230 (2023). doi:10.1145/3626717

[4] Amazon Web Services. 2026. Amazon EC2 On-Demand Pricing. https://aws.amazon.com/ec2/pricing/on-demand/. c6a.48xlarge (192 vCPUs), $7.344 per hour, US East (N. Virginia); accessed September 2026.

[5] Amazon Web Services. 2026. Amazon S3 Pricing. https://aws.amazon.com/s3/pricing/. S3 Standard, first 50 TB/month, US East (N. Virginia); accessed September 2026.

[6] Vijay Arya, Naveen Garg, Rohit Khandekar, Adam Meyerson, Kamesh Munagala, and Vinayaka Pandit. 2004. Local Search Heuristics for k-Median and Facility Location Problems. *SIAM J. Comput.* 33, 3 (2004), 544–562.

[7] Souvik Bhattacherjee, Amit Chavan, Silu Huang, Amol Deshpande, and Aditya G. Parameswaran. 2015. Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1346–1357. doi:10.14778/2824032.2824035

[8] Moses Charikar, Kevin Chen, and Martin Farach-Colton. 2002. Finding Frequent Items in Data Streams. In *International Colloquium on Automata, Languages and Programming (ICALP)*.

[9] Yann Collet and Murray Kucherawy. 2021. Zstandard Compression and the 'application/zstd' Media Type. RFC 8878.

[10] Jarek Duda. 2013. Asymmetric Numeral Systems: Entropy Coding Combining Speed of Huffman Coding with Compression Rate of Arithmetic Coding. arXiv:1311.2540.

[11] Jack Edmonds. 1967. Optimum Branchings. *J. Res. Nat. Bur. Standards* 71B, 4 (1967), 233–240.

[12] Gemma Team. 2024. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118.

[13] Anxin Guo, Jingwei Li, Pattara Sukprasert, Samir Khuller, Amol Deshpande, and Koyel Mukherjee. 2024. To Store or Not to Store: A Graph Theoretical Approach for Dataset Versioning. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. arXiv:2402.11741.

[14] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. 2011. Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions. *SIAM Rev.* 53, 2 (2011), 217–288.

[15] Wei Hao, Daniel Mendoza, Rafael Mendes, Deepak Narayanan, Amar Phanishayee, Asaf Cidon, and Junfeng Yang. 2024. MGit: A Model Versioning and Management System. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (PMLR, Vol. 235)*. 17597–17615.

[16] Anat Heilper and Doron Singer. 2025. Lossless Compression of Neural Network Components: Weights, Checkpoints, and K/V Caches in Low-Precision Formats. arXiv:2508.19263.

[17] Moshik Hershcovitch, Andrew Wood, Leshem Choshen, Guy Girmonsky, Roy Leibovitz, Ilias Ennmouri, Michal Malka, Peter Chin, Swaminathan Sundararaman, and Danny Harnik. 2025. ZipNN: Lossless Compression for AI Models. In *IEEE International Conference on Cloud Computing (CLOUD)*. arXiv:2411.05239.

[18] Kaisei Hishida, Chunwei Liu, John Paparrizos, and Aaron J. Elmore. 2025. Beyond Compression: A Comprehensive Evaluation of Lossless Floating-Point Compression. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4396–4409. doi:10.14778/3749646.3749701

[19] Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. Charting and Navigating Hugging Face's Model Atlas. arXiv:2503.10633.

[20] Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2025. Unsupervised Model Tree Heritage Recovery. In *International Conference on Learning Representations (ICLR)*.

[21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.

[22] Silu Huang, Liqi Xu, Jialin Liu, Aaron J. Elmore, and Aditya G. Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proceedings of the VLDB Endowment* 10, 10 (2017).

[23] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.

[24] Hugging Face. [n. d.]. Xet Chunk-Level Deduplication. https://huggingface.co/docs/hub/xet/deduplication. Accessed 2026-09-24.

[25] Yuriy Kim and Evgeny Belyaev. 2025. An Efficient Compression of Deep Neural Network Checkpoints Based on Prediction and Context Modeling. In *IEEE NW Russia Young Researchers in Electrical and Electronic Engineering Conference (EIConRusNW)*. arXiv:2506.12000.

[26] Tingfeng Lan, Zirui Wang, Yunjia Zheng, Zhaoyuan Su, Juncheng Yang, and Yue Cheng. 2026. TensorDex: A Compact, Tensor-Centric Storage System for Modern AI Models. In *Proceedings of the ACM SIGOPS 32nd Symposium on Operating Systems Principles (SOSP)*. arXiv:2604.17104.

[27] Ruiyuan Li, Zheng Li, Yi Wu, Chao Chen, and Yu Zheng. 2023. Elf: Erasing-based Lossless Floating-Point Compression. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1763–1776. doi:10.14778/3587136.3587149

[28] Wenshuo Li, Xinghao Chen, Han Shu, Yehui Tang, and Yunhe Wang. 2024. ExCP: Extreme LLM Checkpoint Compression via Weight-Momentum Joint Shrinking. In *International Conference on Machine Learning (ICML)*.

[29] Panagiotis Liakos, Katia Papakonstantinopoulou, and Yannis Kotidis. 2022. Chimp: Efficient Lossless Floating Point Compression for Time Series Databases. *Proceedings of the VLDB Endowment* 15, 11 (2022), 3058–3070. doi:10.14778/3551793.3551852

[30] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Conference on Machine Learning and Systems (MLSys)*.

[31] James Liu, Guangxuan Xiao, Kai Li, Jason D. Lee, Song Han, Tri Dao, and Tianle Cai. 2024. BitDelta: Your Fine-Tune May Only Be Worth One Bit. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[32] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.

[33] Michael Maddox, David Goehring, Aaron J. Elmore, Samuel Madden, Aditya G. Parameswaran, and Amol Deshpande. 2016. Decibel: The Relational Dataset Branching System. *Proceedings of the VLDB Endowment* 9, 9 (2016), 624–635.

[34] Hui Miao, Ang Li, Larry S. Davis, and Amol Deshpande. 2017. Towards Unified Data and Lifecycle Management for Deep Learning. In *IEEE International Conference on Data Engineering (ICDE)*. 571–582.

[35] Wanyi Ning, Jingyu Wang, Qi Qi, Mengde Zhu, Haifeng Sun, Daixuan Cheng, Jianxin Liao, and Ce Zhang. 2024. FM-Delta: Lossless Compression for Storing Massive Fine-tuned Foundation Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[36] Tuomas Pelkonen, Scott Franklin, Paul Cavallaro, Qi Huang, Justin Meza, Justin Teller, and Kaushik Veeraraghavan. 2015. Gorilla: A Fast, Scalable, In-Memory Time Series Database. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1816–1827. doi:10.14778/2824032.2824078

[37] Bowen Ping, Shuo Wang, Hanqing Wang, Xu Han, Yuzhuang Xu, Yukun Yan, Yun Chen, Baobao Chang, Zhiyuan Liu, and Maosong Sun. 2024. Delta-CoMe: Training-Free Delta-Compression with Mixed-Precision for Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[38] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115.

[39] Manasi Vartak, Joana M. F. da Trindade, Samuel Madden, and Matei Zaharia. 2018. MISTIQUE: A System to Store and Query Model Intermediates for Model Diagnosis. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. doi:10.1145/3183713.3196934

[40] Manasi Vartak, Harihar Subramanyam, Wei-En Lee, Srinidhi Viswanathan, Saadiyah Husnoo, Samuel Madden, and Matei Zaharia. 2016. ModelDB: A System for Machine Learning Model Management. In *Workshop on Human-In-the-Loop Data Analytics (HILDA@SIGMOD)*.

[41] Daniel Waddington and Cornel Constantinescu. 2025. Lossless Compression for LLM Tensor Incremental Snapshots. arXiv:2505.09810.

[42] Zirui Wang, Tingfeng Lan, Zhaoyuan Su, Juncheng Yang, and Yue Cheng. 2026. ZipLLM: Efficient LLM Storage via Model-Aware Synergistic Data Deduplication and Compression. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. arXiv:2505.06252.

[43] Xiaozhe Yao, Qinghao Hu, and Ana Klimovic. 2025. DeltaZip: Efficient Serving of Multiple Full-Model-Tuned LLMs. In *European Conference on Computer Systems (EuroSys)*. doi:10.1145/3689031.3717468

[44] Tianyi Zhang, Mohsen Hariri, Shaochen Zhong, Vipin Chaudhary, Yang Sui, Xia Hu, and Anshumali Shrivastava. 2025. 70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11). In *Advances in Neural Information Processing Systems (NeurIPS)*.